USENIX WOOT '26 Artifact Appendix: HotWire: Real-World Impersonation and Discharge Attacks on Electric Vehicle Charging Systems

Kuan Yu Chen
National Taiwan University of Science and Technology

A Artifact Appendix

A.1 Abstract

HotWire is a bidirectional testbed for the DIN 70121 and ISO 15118-2 vehicle-to-grid (V2G) charging protocols. It implements both sides of the protocol — a rogue Electric Vehicle Supply Equipment (EVSE, charging station) and a rogue Plug-in Electric Vehicle (PEV) — plus two concrete attacks documented in the paper: *(A1) Autocharge EVCCID impersonation*, in which a fabricated SessionSetupReq causes a production charging station to authorize a session under a victim's billing account; and *(A2) Forced discharge via fabricated PreChargeRes voltage claims*, in which crafted protocol responses cause a vulnerable BMS to close its high-voltage contactors against an attacker-controlled load.

The artifact contains the full Python implementation, a Docker-based continuous-integration test harness (240 unit and integration tests passing), parametric simulation scripts that exercise the DIN 70121, ISO 15118-2, and Tesla protocol variants, hardware design and recovery documentation, safety notes, and historical Luxgen N7 forced-discharge bench logs cited as A2 evidence. We request the **Artifacts Available** and **Artifacts Functional** badges. We do not request **Results Reproduced**: paper §6 evaluates attacks against production charging infrastructure and four NDA-covered vehicles that the AEC cannot reasonably access. Framework-level results in paper §5 (parametric matrix, 32 runs) are covered by the Functional-badge path described in A.4.

A.2 Description & Requirements

The artifact bundles all source code, build files, test harness, documentation, and a one-command verification script. It can be evaluated end-to-end inside Docker plus IPv6 `::1` loopback; no real charging station, no real vehicle, and no PowerLine Communication (PLC) modem are required for any evaluation step.

A.2.1 Security, privacy, and ethical concerns

This artifact contains live offensive-security code. Two attack playbooks are shipped:

- `hotwire/attacks/autocharge_impersonation.py` crafts a SessionSetupReq carrying an arbitrary EVCCID. If executed against a real public charging station whose Autocharge feature trusts EVCCID without further authentication, the station will authorize the session under whatever billing account is linked to that EVCCID, producing unauthorized financial transactions.
- `hotwire/attacks/forced_discharge.py` crafts PreChargeRes / CurrentDemandRes messages with fabricated voltage readings. If executed against a real vehicle without a safety load bank, current-limiting interlocks, and emergency disconnect, the target BMS may close its contactors against a 0 V (or attacker-controlled) load, causing uncontrolled battery discharge.

All evaluation paths defined in A.3 and A.4 are safe for the AEC: they execute entirely inside a Docker container or on `::1` IPv6 loopback. No attack frame leaves the evaluator's host. The hardware-aware scripts under `scripts/hw_check/phase[234]_*.py` that *do* emit real CCS protocol frames are not part of the AEC evaluation path; the AEC should not run them.

Full safety, authorization, and disclosure documentation is in `SAFETY.md`, `SECURITY.md`, and `docs/ethics_evidence.md` in the artifact root.

A.2.2 How to access

The archived artifact is on Zenodo at:

`https://doi.org/10.5281/zenodo.19754105`

The source tree is also browsable at the public GitHub repository. Per the USENIX requirement for a stable reference rather than an evolving repository pointer, the evaluated version corresponds to commit `03623bf` on branch `main`:

`https://github.com/sickcell6000/HotWire/tree/03623bfcd89cbccccfa4d657ea200fe14888b6e2`

A camera-ready stable reference will be posted as a Zenodo new version after the discussion period concludes, and the reference in this appendix updated accordingly.

A.2.3 Hardware dependencies

For the AEC evaluation path: **None.** Any host with at least 4 GB RAM, 10 GB free disk, and a 64-bit OS that runs Docker Desktop / Docker Engine is sufficient. Docker Desktop on Windows or macOS works; native Docker on Linux works.

The paper's real-world experiments used: a Raspberry Pi 4 (4 GB) plus a TP-Link TL-PA4010 modified per `docs/hardware_design_guide.md`, plus a Windows 11 host with a similarly modified TL-PA4010. The AEC does not need this hardware.

A.2.4 Software dependencies

For the Functional-badge path:

- Docker 24+ with Compose v2 (Docker Desktop or native Linux docker-ce). Verified on Docker 29.1.3.
- No additional packages on the host. The Docker image bundles Python 3.12, PyQt6, the OpenV2Gx EXI codec built from source, `pcap-ct`, `psutil`, `pytest`, and the rest of `requirements.txt`.

A non-Docker fallback exists for hosts where Docker is unavailable: Python 3.9–3.12, plus `pip install -r requirements.txt`. The Functional-badge path runs equivalently in either mode.

A.2.5 Benchmarks

None. HotWire is a protocol implementation and attack framework, not a benchmark suite. The parametric simulation matrix (F3 below) is self-contained: it generates its own input data (target voltages, durations, protocol variants) deterministically from the script.

A.3 Set-up

A.3.1 Installation

1. Download the artifact from the Zenodo DOI in A.2.2.
2. Extract the archive: `unzip HotWire-artifact.zip && cd HotWire`.
3. Confirm Docker is reachable: `docker info | grep Server:` should show `Server: Docker Engine - <something>`. On Windows / macOS, open Docker Desktop first and wait for "Engine running".
4. (Optional, saves ~20 min) load the pre-built Docker image from the bundled tarball: `docker load -i hotwire-ci.tar.gz`. If skipped, the image is built on first run.

A.3.2 Basic Test

Run:

```
docker compose run --rm hotwire-ci
```

Expected output (final lines):

```
240 passed, 6 skipped in ~62s
>>> [regression] OK
Overall exit code: 0
```

If those lines appear, all four CI phases (codec, hw_check, gui_smoke, regression) have passed and the artifact is functional.

A.4 Evaluation workflow

A.4.1 Major Claims

We request the Artifacts Functional badge against the following framework-level claims (Cx). The Results Reproduced badge is *not* requested; paper §6 real-world claims depend on infrastructure unavailable to the AEC.

(C1)
HotWire's protocol stack passes a 240-test pytest regression suite covering DIN 70121 message encoding, SLAC, SDP, EVSE/PEV state machines, and attack payload construction. Established by experiment **(E1)**, paper §5 Table *(test matrix)*.

(C2)
HotWire walks a complete DIN 70121 session end-to-end in pure software — 13 PEV state transitions from Connecting through sustained CurrentDemand — on a single host using IPv6 loopback. Established by **(E2)**, paper §5.

(C3)
The framework is stable across the parameter range declared in the paper (target voltages 200/400/800 V, session durations 10/25/45 s). Established by **(E3)**, paper §5.

(C4)
The two attack payloads documented in the paper exist as inspectable Python source in the artifact. Established by **(E4)**.

A.4.2 Experiments

A single wrapper script `./verify_artifact.sh` runs all four experiments and prints a colored pass/fail verdict.

(E1) Docker CI regression
[5 human-minutes + 1 compute-minute + 800 MB disk if pre-built]: confirms (C1).

Preparation: ensure Docker daemon is reachable. **Execution:** `docker compose run --rm hotwire-ci`. **Results:** look for the line `240 passed, 6 skipped` (or higher if PyQt6 is available outside Docker too) followed by `>>> [regression] OK`. The exit code must be 0. Coverage HTML is written to `htmlcov/`; JUnit XML to `reports/`.

(E2) Simulation-mode full V2G session
[2 human-minutes + 30 compute-seconds]: confirms (C2).

Preparation: ensure Python 3.9–3.12 is available (`python3 --version`). **Execution:** `bash scripts/sim_loopback.sh 25`. **Results:** the script prints PEV state transitions and a final summary. Confirm at least 13 distinct `entering N:Stage` lines and at least 5 `CurrentDemandReq` cycles. The end-of-run banner shows PEV final state `Charging` at simulated 400 V / 50 A.

(E3) Parametric matrix (9 runs)
[2 human-minutes + 5 compute-minutes]: confirms (C3).
Preparation: same as (E2). **Execution:** `bash scripts/sim_matrix.sh`. **Results:** the script prints a 9-row table covering 3 target voltages × 3 durations. All 9 rows must have a `PASS` column entry. Reference output: `expected_outputs/sim_matrix.log`.

(E4) Attack code presence and validity
[1 human-minute + 5 compute-seconds]: confirms (C4).
Preparation: none. **Execution:**

```
python3 -m py_compile \
  hotwire/attacks/autocharge_impersonation.py
python3 -m py_compile \
  hotwire/attacks/forced_discharge.py
```

Results: both commands exit with status 0 and produce no output. The two files exist and parse as valid Python; the AEC may inspect their source to confirm the fabricated-frame builders match the paper's protocol-level descriptions.

After all four experiments, the wrapper script prints `[verify_artifact] ✓ ALL FUNCTIONAL CHECKS PASSED` and exits 0. Total wall-clock: ~5 minutes with the pre-loaded Docker image, ~25 minutes if building from source.

A.5 Notes on Reusability

Beyond the AEC evaluation, the artifact may be reused for:

- *Defensive validation:* EV manufacturers and charging network operators can use the simulation harness to verify that their own stack rejects malformed or out-of-sequence protocol messages. The parametric matrix is the natural extension point; add new rows to test additional voltage / current / SoC operating points.
- *Protocol research:* the bidirectional EVSE+PEV simulator is independent from the attack code. To study a different DIN/ISO message exchange, modify or subclass `hotwire/fsm/fsm_pev.py` or `hotwire/fsm/fsm_evse.py`.
- *Hardware bring-up:* `docs/hardware_design_guide.md` documents the physical CCS coupling network, modem programming sequence, and a Level 0–7 recovery procedure for stuck modems. This is generic to any QCA7420-based research, not just HotWire.
- *Cross-platform CCS testing:* HotWire runs natively on Linux (verified on Raspberry Pi OS 12, Ubuntu 24.04, Debian 12) and on Windows 11 with Npcap. The `patches/pyplc/` directory publishes the three patches required for upstream pyPLC to also run on Windows.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20231005. Submission, reviewing, and badging methodology followed for the evaluation of this artifact can be found at `https://secartifacts.github.io/usenixsec2026/`.